Spring: Spock vs. Kotest vs. JUnit - When to Pick Which One?

Marek Dominiak



Marek Dominiak

I run training in EventStorming, Architecture, DDD, and TDD

19 years of professional experience

Co-owner and instructor at trainitek

Hands-on Architect and Team Lead at MINTRA

https://www.linkedin.com/in/marekdominiak

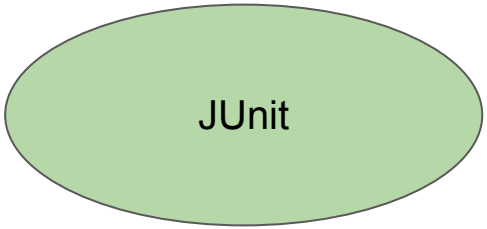
Agenda

- Core Features & Flexibility
- Spring Testing Basics
- Spring Boot & Testcontainers Integration
- Gotchas
- Performance & Developer Experience
- Decision Guide
- Q&A

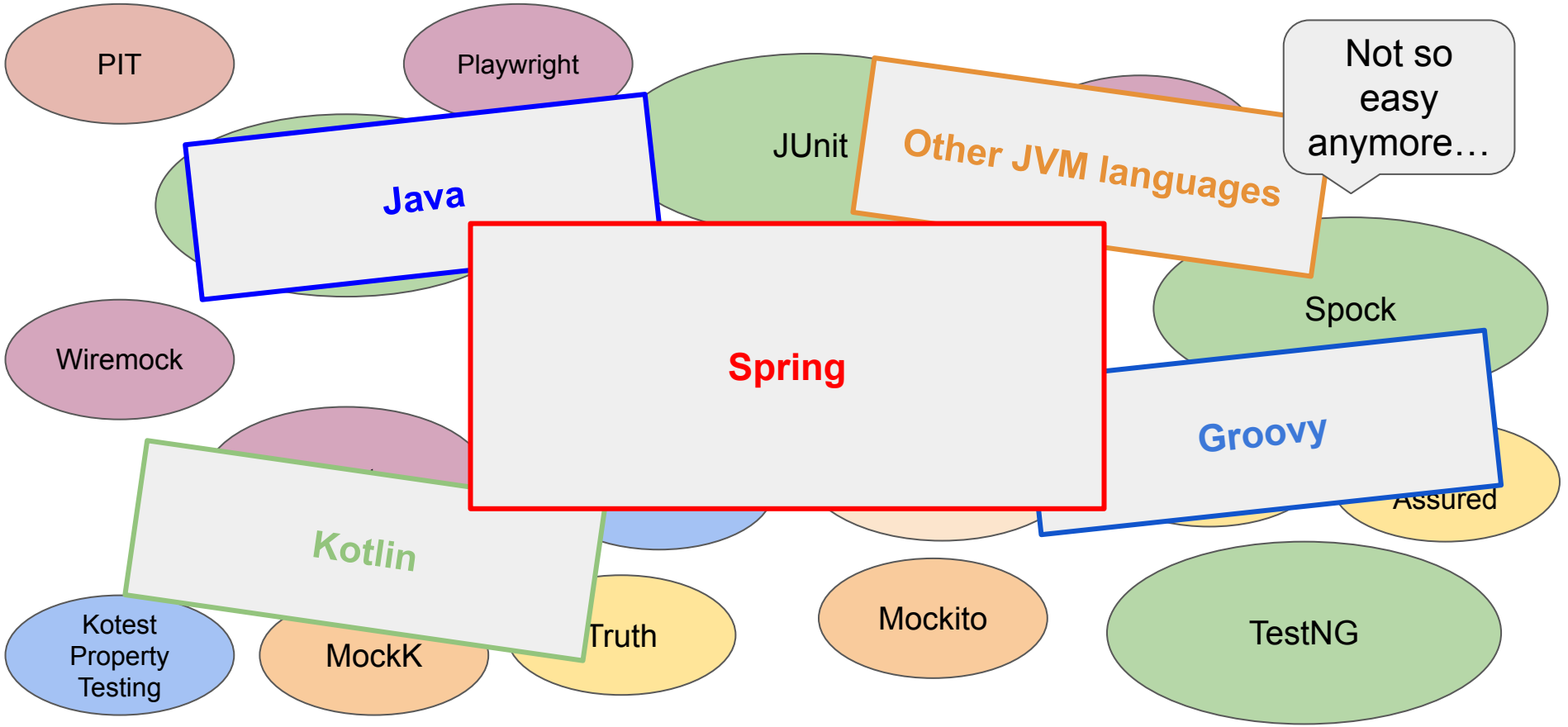


Why do we even talk about choosing testing library?

At least in ~2000s :)

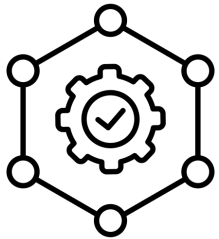


Why do we even talk about choosing testing library?



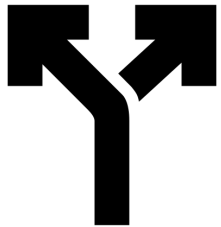
Testing Frameworks

Framework	Started	Current
JUnit	**1997**	**6.1.3**
Spock	**2008**	**2.4**
Kotest	**2016** as KotlinTest	**6.2.3**



Alternatives

- JUnit on Java
- Spock
- Kotest
- JUnit on Kotlin



What we compare

- Language & test structure
- Assertions
- Mocking
- Test tables (data-driven)
- Spring integration
- Performance
- Ecosystem & tooling



Core Features & Flexibility

Given / When / Then

- Let’s see the Ultimate Test Template in each of the Frameworks

```
@Test
public void shouldDoSomethingCool() throws Exception {
    //given

    //when

    //then

}
```

Ref 1: First introduced: https://dannorth.net/blog/introducing-bdd/
Ref 2: I learned it from: https://szczepiq.wordpress.com/2009/12/07/given-when-then-forever/

Given / When / Then - JUnit

```
class TalkBddTest {

    @Test
    @DisplayName("A talk ends according to its duration")
    void aTalkEndsAccordingToItsDuration() {
        // given: a 45-minute talk starting at 15:40
        Talk talk = aTalk()
                .startingAt(LocalDateTime.of( year: 2026,  month: 9,  dayOfMonth: 3,  hour: 15,  minute: 40))
                .lastingMinutes( durationMinutes: 45)
                .build();

        // when: its end time is requested
        var endsAt = talk.endsAt();

        // then: it ends at 16:25
        assertEquals(LocalDateTime.of( year: 2026,  month: 9,  dayOfMonth: 3,  hour: 16,  minute: 25), endsAt);
    }
}
```

Given / When / Then - JUnit on Kotlin

```kotlin
class TalkBddKotlinTest {  1 Usage

    @Test
    fun `a talk ends according to its duration`() {
        // given: a 45-minute talk starting at 15:40
        val talk = aTalk()
            .startingAt( startsAt = LocalDateTime.of( year = 2026,  month = 9,  dayOfMonth = 3,  hour = 15,  minute = 40))
            .lastingMinutes( durationMinutes = 45)
            .build()

        // when: its end time is requested
        val endsAt = talk.endsAt()

        // then: it ends at 16:25
        assertEquals( expected = LocalDateTime.of( year = 2026,  month = 9,  dayOfMonth = 3,  hour = 16,  minute = 25),  actual = endsAt)
    }
}
```

Given / When / Then - Spock

```
class TalkBddSpec extends Specification {

    def 'a talk ends according to its duration'() {
        given: 'a 45-minute talk starting at 15:40'
        def talk : Talk  = aTalk()
                .startingAt(LocalDateTime.of( year: 2026,  month: 9,  dayOfMonth: 3,  hour: 15,  minute: 40))
                .lastingMinutes( durationMinutes: 45)
                .build()

        when: 'its end time is requested'
        def endsAt : LocalDateTime  = talk.endsAt()

        then: 'it ends at 16:25'
        endsAt == LocalDateTime.of( year: 2026,  month: 9,  dayOfMonth: 3,  hour: 16,  minute: 25)
    }
}
```

Given / When / Then - Kotest #1

```
class TalkBddSpec : BehaviorSpec( body = {  2 Usages  new *
    given( name = "a 45-minute talk starting at 15:40") {
        val startsAt = LocalDateTime.of( year = 2026,  month = 9,  dayOfMonth = 3,  hour = 15,  minute = 40)
        val talk = aTalk().startingAt(startsAt).lastingMinutes( durationMinutes = 45).build()

        `when`( name = "its end time is requested") {
            val endsAt = talk.endsAt()

            then( name = "it ends at 16:25") {
                endsAt shouldBe LocalDateTime.of( year = 2026,  month = 9,  dayOfMonth = 3,  hour = 16,  minute = 25)
            }
        }
    }
})
```

Given / When / Then - Kotest #2

```
class TalkBddFunSpec : FunSpec( body = {  1 Usage
    test( name = "a talk ends according to its duration") {
        // given: a 45-minute talk starting at 15:40
        val talk = aTalk()
            .startingAt( startsAt = LocalDateTime.of( year = 2026,  month = 9,  dayOfMonth = 3,  hour = 15,  minute = 40))
            .lastingMinutes( durationMinutes = 45)
            .build()

        // when: its end time is requested
        val endsAt = talk.endsAt()

        // then: it ends at 16:25
        endsAt shouldBe LocalDateTime.of( year = 2026,  month = 9,  dayOfMonth = 3,  hour = 16,  minute = 25)
    }
})
```

Kotest has 10 different flavours!
Fun Spec, String Spec, Should Spec, Describe Spec, Behavior Spec, Word Spec, Free Spec, Feature Spec, Expect Spec, Annotation Spec

Assertions - JUnit

JUnit

```
assertAll(
        () -> assertEquals(title, talk.title()),
        () -> assertEquals(ACCEPTED, talk.status()),
        () -> assertEquals(startsAt.plusMinutes(duration), talk.endsAt()),
        () -> assertEquals(room, talk.room())
);
```



```
org.opentest4j.AssertionFailedError:
Expected :2026-09-03T16:26
Actual   :2026-09-03T16:25
<Click to see difference>

 <1 internal line>
    at com.trainitek.conference.TalkDomainTest.calculatesTheEndOfAnAcceptedTalk
     (TalkDomainTest.java:65)
```

Assertions - Kotest

Kotest

```
assertSoftly(talk) {
    title shouldBe expectedTitle
    status shouldBe ACCEPTED
    endsAt() shouldBe startsAt.plusMinutes(duration.toLong())
    room shouldBe expectedRoom
}
```



```
org.opentest4j.AssertionFailedError: The following assertion for
Talk[id=TalkId[value=3f185845-9202-41f7-af5e-b4950e7b30ad], title=Spring: Spock vs. Kotest
vs. JUnit - When to Pick Which One?, speaker=Marek Dominiak, status=ACCEPTED,
durationMinutes=45, room=Room 3, startsAt=2026-09-03T15:40] failed:
expected:<2026-09-03T16:26> but was:<2026-09-03T16:25> <5 internal lines>
```

Assertions - Spock

Spock

```
verifyAll(talk) { Talk it ->
    title() == expectedTitle
    status() == ACCEPTED
    endsAt() == startsAt.plusMinutes(duration)
    room() == expectedRoom
}
```



```
Condition not satisfied:

endsAt() == startsAt.plusMinutes(duration + 1)
|        |  |        |           |        |
|        |  |        |           45       46
|        |  |        2026-09-03T16:26
|        |  2026-09-03T15:40
|        false
2026-09-03T16:25
```

Assertions

- **JUnit**: basic messages, **AssertJ** improves them
- **Spock**: shows every value in the failed condition!
- **Kotest**: clear matcher messages and useful diffs
- Most detailed out of the box: **Spock**



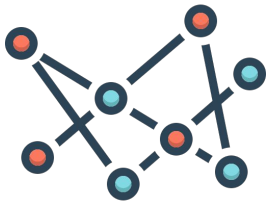
Lifecycle methods - comparison

Framework	Before Class	Before Test	After Test	After Class
JUnit	`@BeforeAll` - once, before all tests in the class	`@BeforeEach` - before each test method	`@AfterEach` - after each test method	`@AfterAll` - once, after all tests in the class
Spock	`setupSpec()` - once, before all feature methods	`setup()` - before each feature method (test)	`cleanup()` - after each feature method (test)	`cleanupSpec()` - once, after all feature methods
Kotest	`beforeSpec { }` - once, before all tests in the spec	`beforeEach { ctx -> }` - before each test case	`afterEach { ctx -> }` - after each test case	`afterSpec { }` - once, after all tests in the spec

Note: Kotest adds extra granularity (e.g. container-level `beforeAny` and `beforeTest` beyond what JUnit or Spock provide.

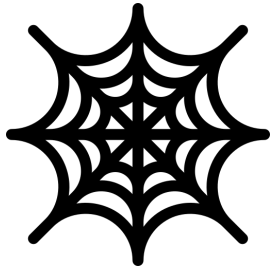
Mocking and verification

- **JUnit** + **Mockito**: *`verify(mock).call()`*
- **Spock**: mocking built in; *`1 * mock.call()`*
- **Kotest** + **MockK**: *`verify { mock.call() }`*
- **JUnit on Kotlin** + **MockK**: *`verify { mock.call() }`*



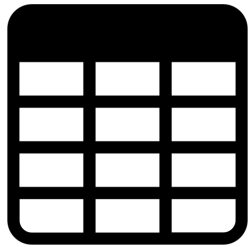
Nesting tests

- **JUnit**: optional via *@Nested*, flat tests more common
- **Spock**: no direct equivalent, flat tests are standard
- **Kotest**: built into many styles (!), widely used
- **Keep tests independent**



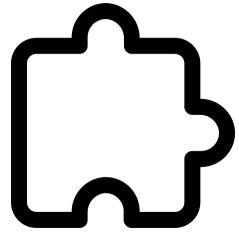
Data tables

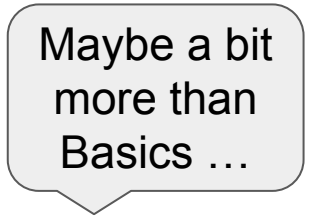
- **JUnit**: *@ParameterizedTest* with a data source
- **Spock**: simple, built-in *where: tables*
- **Kotest**: *withTests*
- This is where **Spock** really shines



Extensions and test utilities

- **JUnit**: mutable clock and polling need extra code/libraries
- **Spock**: *MutableClock* and *PollingConditions* included
- **Kotest**: *eventually* included, *TestClock* needs an extra module
- **Extensions**: supported by all three frameworks





Spring Testing Basics

Connecting tests to Spring

Framework	How to connect it to Spring
JUnit	Use annotations like `@SpringBootTest`, or any slices like `@JdbcTest`, `@JpaTest`.
Spock	As for JUnit, and requires **spock-spring** dependency.
Kotest	As for JUnit, requires **kotest-extensions-spring** and explicit `SpringExtension` registration.

Spring Test contexts

Trade-offs … as always

TalkControllerTest

`@SpringBootTest(RANDOM_PORT)`

- HTTP server
- Controllers
- Services
- Repositories
- Security
- Database
- ...

Kind of slow

Complex

Tests a LOT

TalkControllerWebMvcSliceTest

`@WebMvcTest(TalkController.class)`

- MVC infrastructure
- Controller
- Mocked dependencies

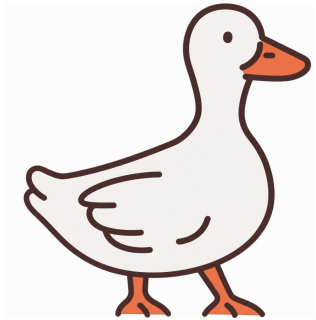
Tests little

Fast

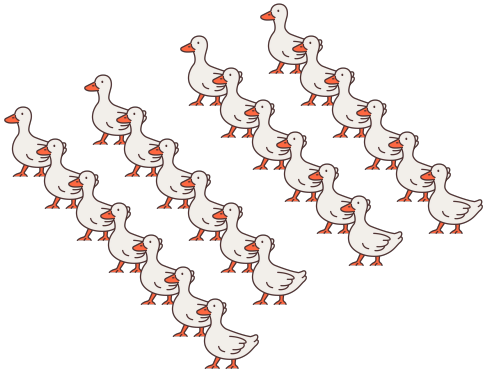
Context is the real cost!

- Starting a new Spring context is **expensive**
- **The same context** can be **cached** and **reused**
- Many small contexts may cost more than one larger reused context

- **The number of contexts often matters more than their size**

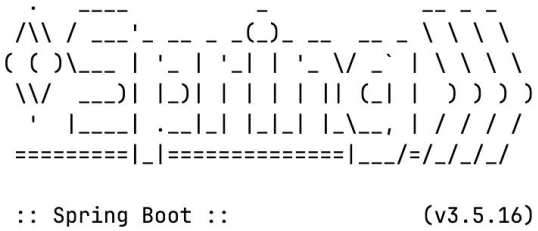


1 horse-sized context **vs** **50 duck-sized contexts**



How do we count Spring Test Contexts?

- Counting Spring logos in logs worked... once 😉
- Better: count created `ApplicationContext`s
- Simple approach: extend `ApplicationContextInitializer`
- Demo: `BenchmarkContextInitializer`



When does Spring create a new context?

When tests differ in:

- Configuration / test type
- Profiles
- Test or dynamic properties
- Imported config
- Mocked / overridden beans
- Initializers / loaders / parent context

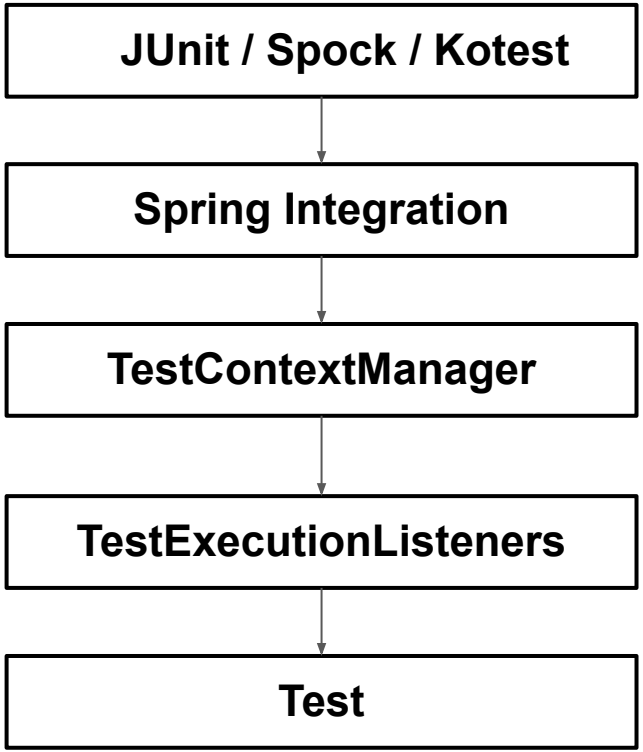


- Also: @DirtiesContext, cache eviction, or a new JVM.

Same config → reuse. Different config → rebuild.

Spring test lifecycle

- Test framework integrates with Spring
- TestContextManager
  runs `TestExecutionListener`s
- They handle:
  - dependency injection
  - transactions and rollback
  - `@Sql`, `@DirtiesContext`, cleanup

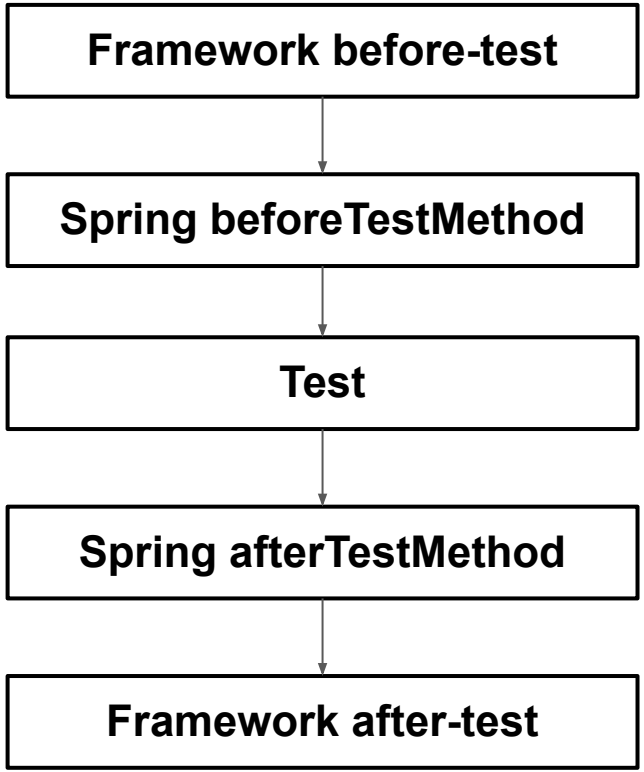


Example: TransactionalTestExecutionListener

Test framework vs Spring test lifecycle

- **JUnit:** one test = one Spring lifecycle
- **Spock**: one test = one Spring lifecycle
- **Kotest**: **it depends ….**
  - Nested tests introduce a **choice**
  - SpringTestLifecycleMode**.Root** **vs** SpringTestLifecycleMode.**Test**

This escalated quickly



Spock / Junit + Spring

Spock

Spring test-method callbacks run around each test

```
def "a test 1" {
    given:

    when:

    then:
}
```

JUnit

Spring test-method callbacks run around each test

```
@Test
void test() {
    // given

    // when

    // then
}
```

Kotest + Spring: Root vs Test lifecycle

Test Execution Events

Spring has various Test Execution Events such as `BeforeTestMethod` and `BeforeTestExecution`.
These events make an assumption that tests are methods, so they do not map exactly one to one to tests defined in frameworks like Kotest where tests are functions and can be nested arbitrarily.

You can customize when these callbacks are fired by using the `SpringTestLifecycleMode` enum when creating the extension. By default, this is on leaf tests. You can set these to fire on root tests by passing a `SpringTestLifecycleMode.Root` argument to the extension:

```
class ProjectConfig : AbstractProjectConfig() {
  override val extensions = listOf(SpringExtension(SpringTestLifecycleMode.Root))
}
```

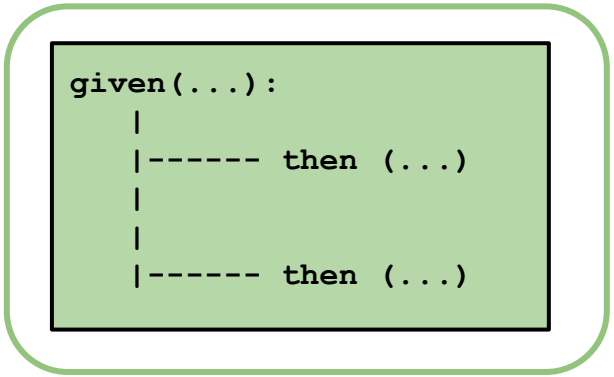
Ref: https://kotest.io/docs/extensions/spring.html#test-execution-events

Kotest + Spring: Root vs Test lifecycle

SpringTestLifecycleMode

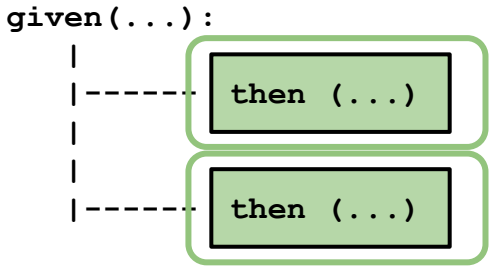
Root

Spring test-method callbacks run around each root test



Test (default)

Spring test-method callbacks run around each leaf test



Ref: https://kotest.io/docs/extensions/spring.html

Testcontainers Integration

Testcontainers & Spring

- Container is a Spring bean
- **`@ServiceConnection`** provides connection details automatically
- Spring Boot configures the DataSource
- Testcontainers runs `schema.sql` when the container starts

Nice!

```
@Bean
@ServiceConnection
PostgreSQLContainer postgres() {
    return new PostgreSQLContainer("postgres:17.10-alpine")
            .withInitScript("schema.sql");
}
```

Spring Contexts matter

- Same Spring context → same container
- New Spring context → new container

Now I am paying attention

Spring Context A

Test 1 --|
Test 2 --|--> **Postgres Container A**
Test 3 --|

Spring Context B

Test 4 --|--> **Postgres Container B**

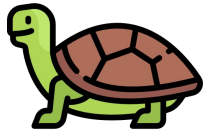
Reuse contexts. Reset database state between tests.

Gotchas



A note about Kotest 5.9

- **Kotest 5.9:** classpath scanning for `@AutoScan` extensions could add noticeable startup cost
- In our project, this made Kotest tests run about 2× slower



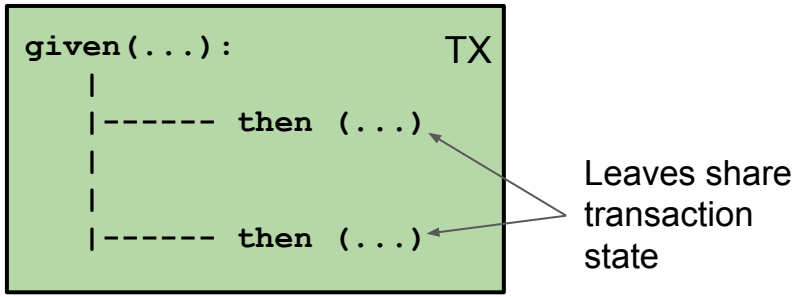
- **Kotest 6.0:** `@AutoScan` / extension classpath scanning was removed
- Extensions are now registered explicitly



Kotest with Spring: @Transactional Gotcha #1

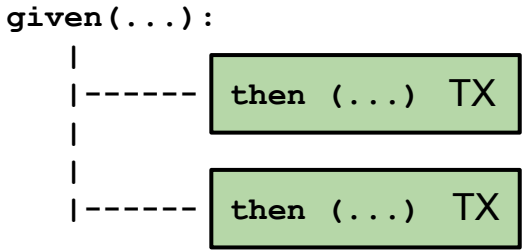
`@Transactional` tests follow the configured Spring lifecycle boundary

Root lifecycle



One transaction per root

Test lifecycle



One transaction per leaf

Kotest with Spring Gotcha #2

- **Root** + **final** class
  - simplest setup, fastest in our benchmark
- **Root + open class via all-open plugin**
  - slightly slower in our benchmark
- **Test** + **open** class
  - per-leaf Spring lifecycle in our setup, slowest in our benchmark
- Some Kotest + Spring setups require an open test class

It's a bit complicated …

We talk benchmarks later on

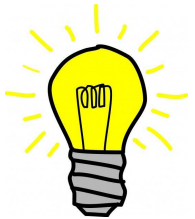
Lifecycle choice can affect both behavior and runtime cost.

Some tips

What worked for us:

- **Root** + **final** + **flat** tests
- **Sequential** execution
- No test-managed @Transactional
- Clean database state between tests

At least for integration tests and web integration tests



More flexibility = more lifecycle combinations to understand

Other Gotchas

JUnit and Spock have their gotchas too.
In this presentation I focused on the ones that matter for **Spring integration**.



Performance & Developer Experience

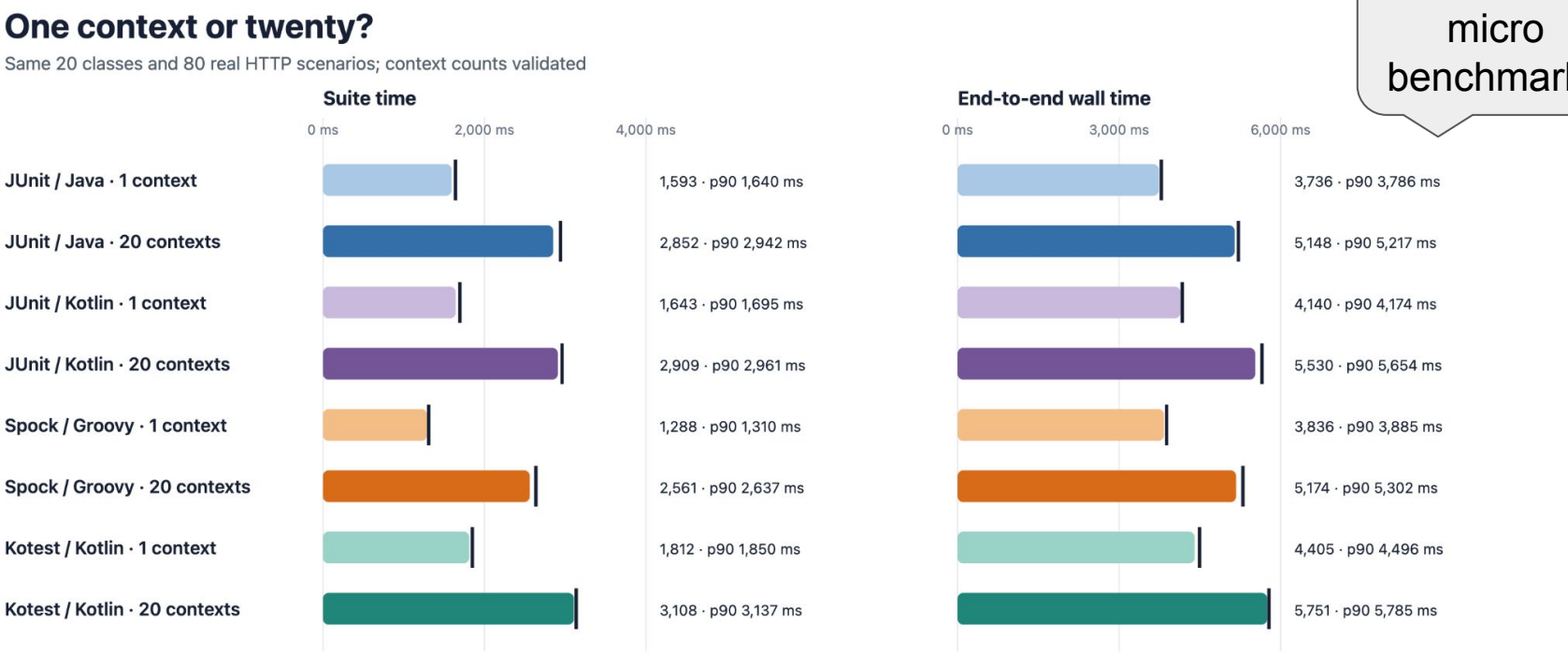
Performance Comparisons #1

Don’t trust micro benchmarks



Performance Comparisons #2

Don’t trust micro benchmarks



Performance Comparisons #3

Don’t trust micro benchmarks



Observations

- Framework overhead is relatively small
- Spring context reuse matters much more
- Kotest Root / final was fastest in our lifecycle benchmark
- Choose lifecycle for semantics first, performance second



Optimize Spring contexts before optimizing the test framework

Decision Guide

Very opinionated decision guide

JUnit

- Default choice for Java
- Also a strong choice for Kotlin
- Simplest Spring integration model

Spock

- Pick it for expressive tests and data tables
- Requires Groovy
- Spring lifecycle stays straightforward



Kotest

- Pick it for Kotlin DSLs and flexibility
- Great support for nested test styles
- More Spring lifecycle choices to understand

What matters most?

- Simplicity + Spring → JUnit
- Data tables + expressive specs → Spock
- Kotlin DSL + flexibility → Kotest
- Performance → optimize Spring contexts first



Q/A

Thank You!

Presentation Slides



marekdominiak



Icons source: https://www.flaticon.com/

Please rate my talk